Pontificia Universidad JAVERIANA Colombia

LLMOps y despliegue

Semana 12 · El asistente de ACME como API
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

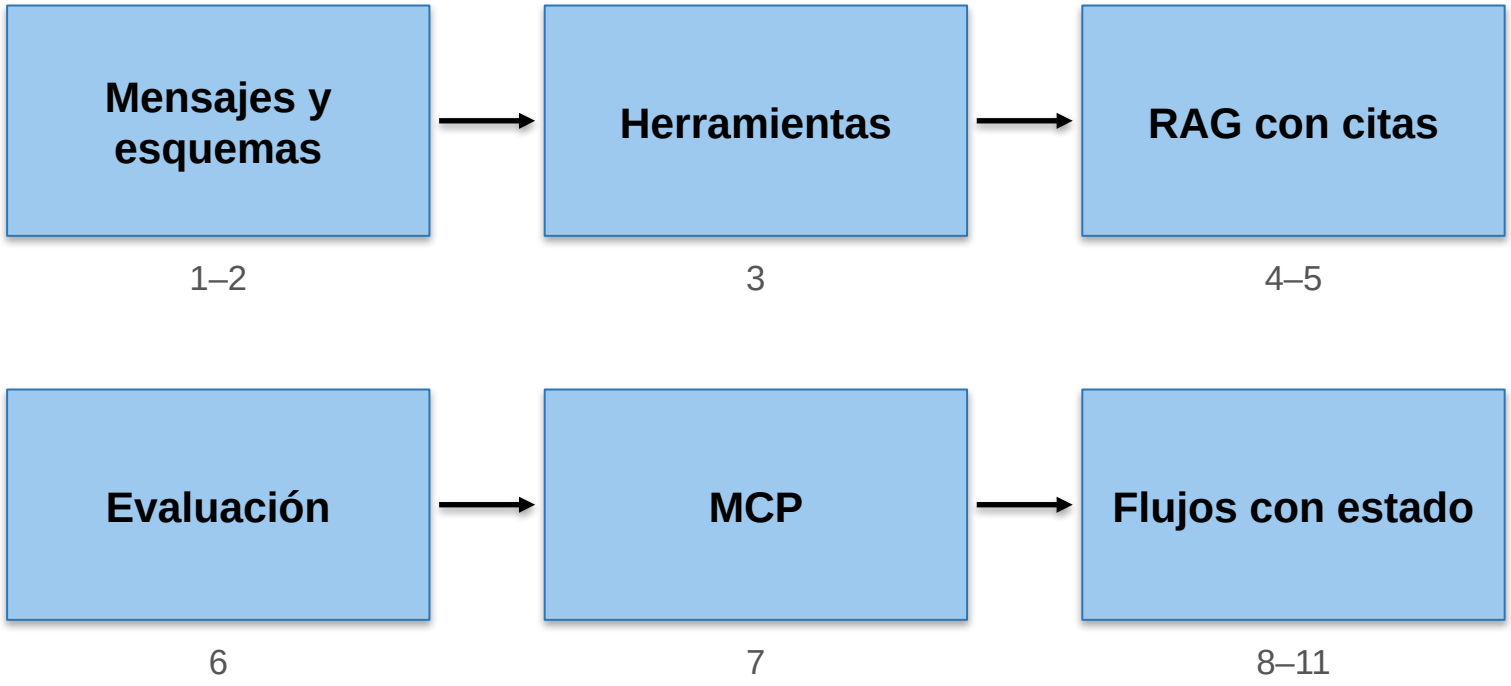
- Fundamentos con LangChain — Semanas 1–3 ✓
- RAG y evaluación — Semanas 4–6 ✓
- Model Context Protocol — Semana 7 ✓
- LangGraph y patrones agénticos — Semanas 8–11 ✓
- **LLMOps y despliegue** — Semana 12

Semana 12 · LLMOps y despliegue

- Del notebook a un servicio
- Un agente que junta las piezas del curso
- FastAPI, `thread_id` y PostgreSQL
- Streaming con SSE
- LangGraph Studio y CI
- Operar: costos, trazas, evaluación y seguridad

Lo que construimos en el curso

Doce semanas, un mismo asistente



Esta semana lo empaquetamos como un servicio que otros pueden usar.

Del notebook a un servicio

El problema de la semana

Qué cambia

1. Muchas personas al tiempo, cada una con su conversación.
2. La memoria debe sobrevivir a un reinicio del servidor.
3. La respuesta debe empezar a mostrarse antes de terminar.
4. Hay que saber cuánto cuesta, qué falló y si la calidad se mantiene.

Notebook frente a servicio

Lo que cambia al desplegar

Aspecto	Notebook	Servicio
Usuarios	Una persona, una celda a la vez	Muchas personas al tiempo
Estado	Variables del kernel	Checkpointer en una base de datos
Errores	Se leen en la celda	Se registran y se responden con un código HTTP
Secretos	`.env` local	Variables del entorno o un gestor de secretos
Costo	Una corrida	Cada turno de cada persona

Pontificia Universidad JAVERIANA Colombia

El agente de ACME

src/agents/acme.py

Un agente con las piezas del curso

src/agents/acme.py

Dos herramientas

search_policies: RAG con citas (semana 4).
check_credit: la cartera del servidor MCP (semana 7).

Cómo funciona

1. El agente de producción se arma con `create_agent`, como en la semana 3.

2. Usa dos herramientas del curso: políticas con citas y créditos del servidor MCP.

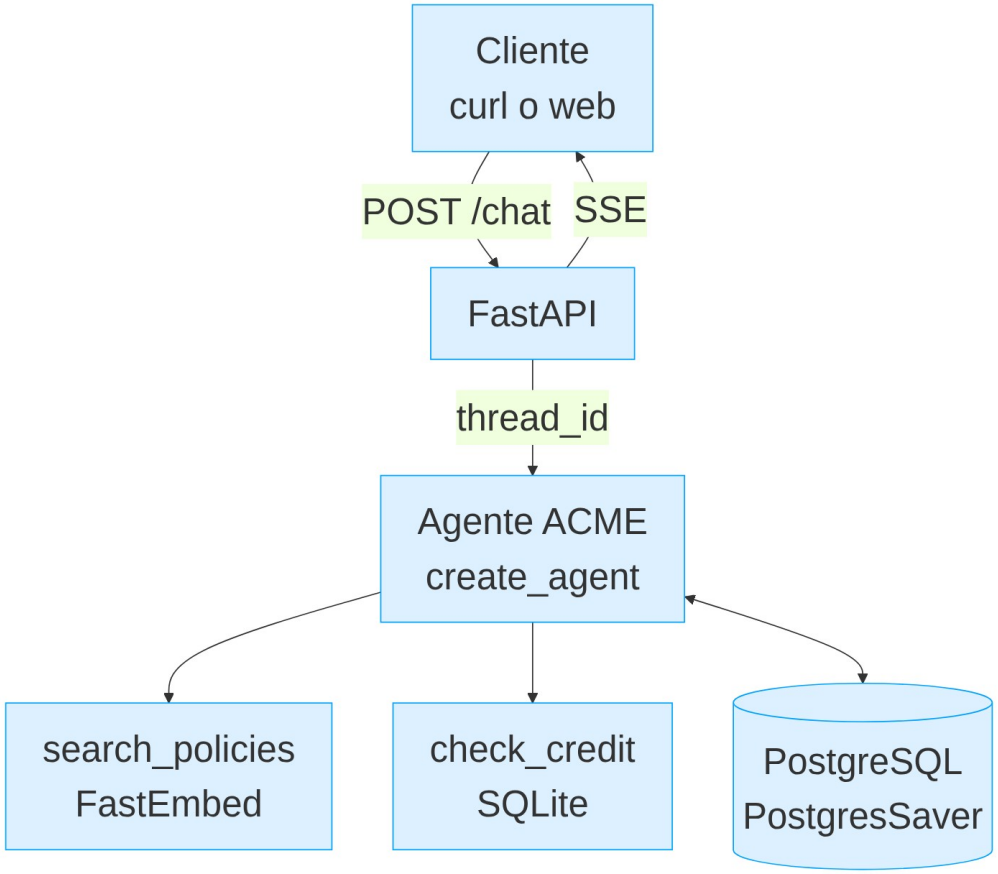
3. El checkpointer se inyecta al desplegar; `langgraph.json` publica el grafo.

La arquitectura

API local con memoria persistente

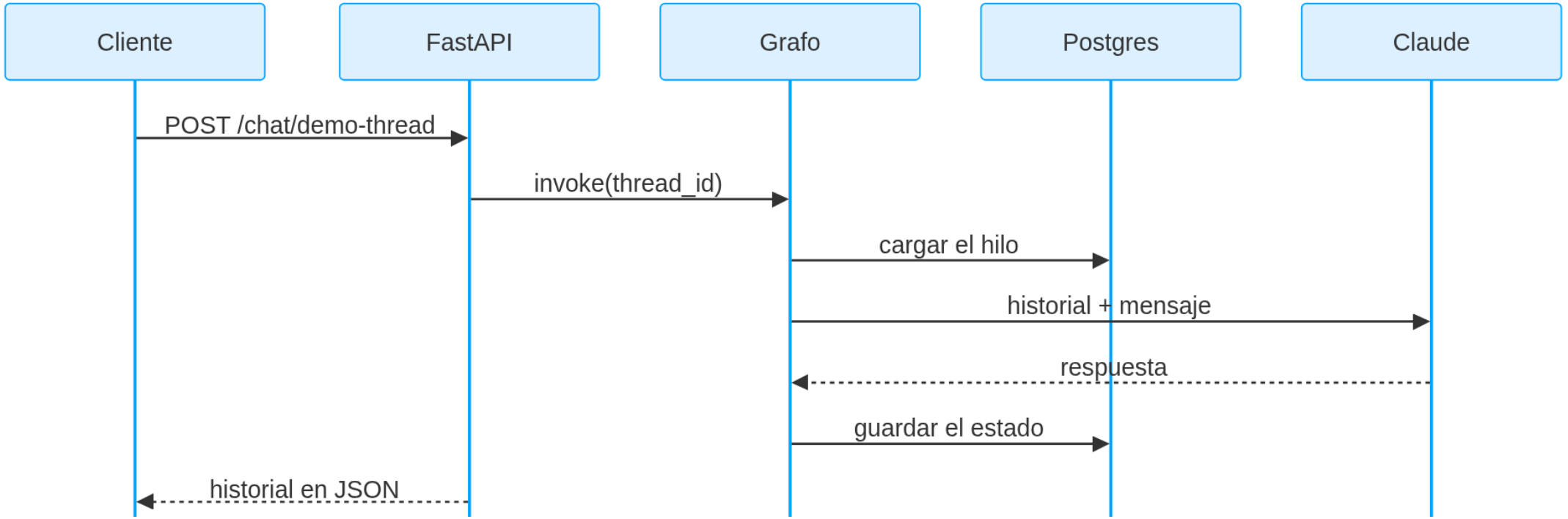
Un hilo por conversación

El `chat_id` de la URL es el `thread_id` del checkpointer.



Un turno de POST /chat

Diagrama de secuencia



Cargar, responder, guardar. El `chat_id` de la URL es el `thread_id`: cada turno carga el hilo, llama al modelo y guarda el estado nuevo.

Los endpoints

src/api/main.py

Endpoint	Qué hace
`GET /health`	Comprueba que el servicio responde
`POST /chat/{chat_id}`	Responde y devuelve el historial guardado
`POST /chat/{chat_id}/stream`	Envía la respuesta por SSE mientras se genera
Mismo `chat_id`	Continúa la conversación, incluso tras reiniciar la API

Probar la API

README · Local support chat API

Dos turnos, un hilo

El segundo turno entiende «lo» porque el hilo está en PostgreSQL.

Cómo funciona

1. Docker levanta PostgreSQL y la API se sirve en el puerto 8000.

2. Un primer mensaje pregunta por el saldo vencido de un cliente.

3. Un segundo mensaje, en el mismo hilo, se responde por streaming y recuerda el primero.

Streaming con SSE

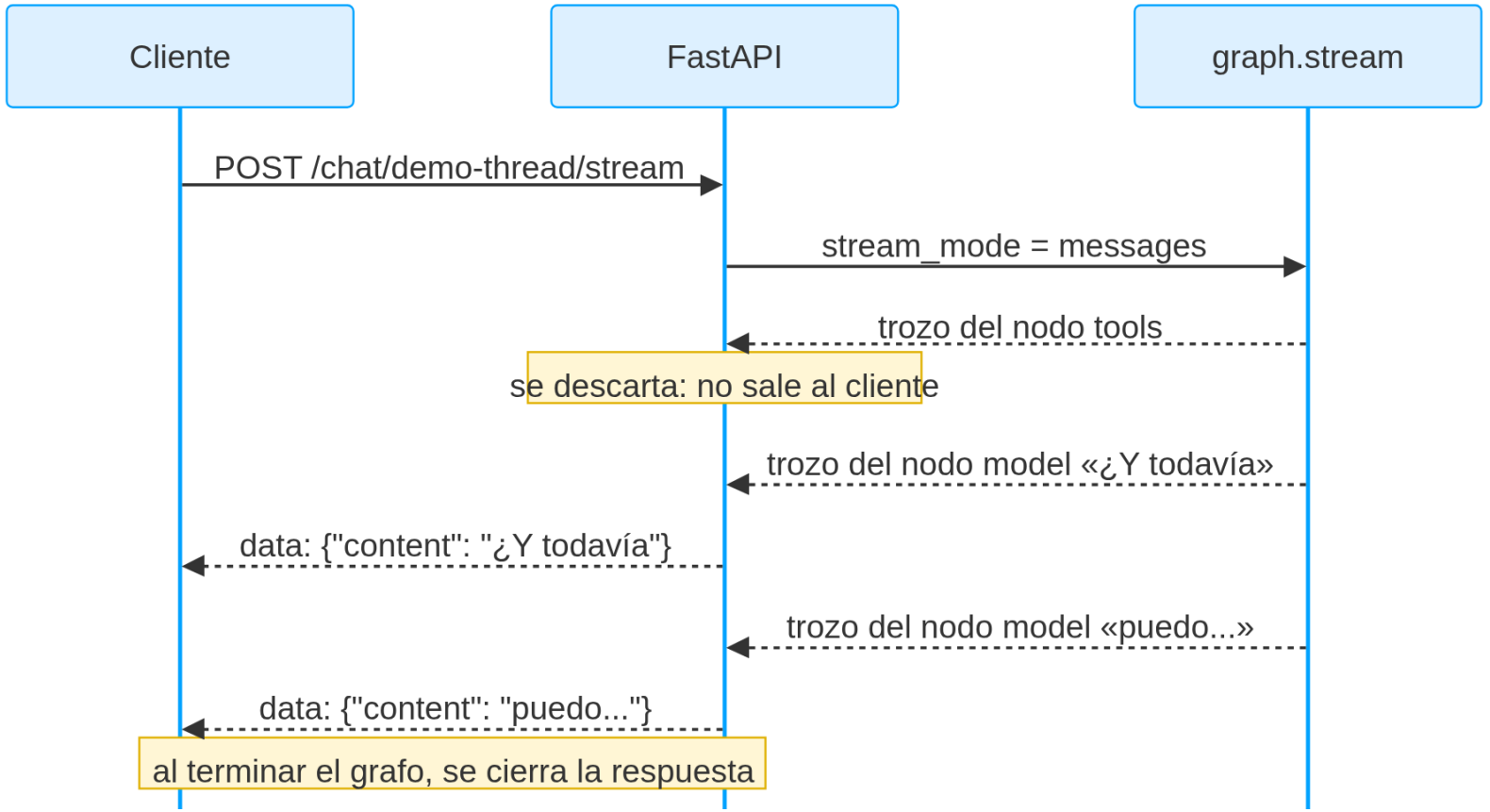
src/api/main.py

Mostrar antes

1. `stream_mode="messages"` entrega los fragmentos del modelo mientras se generan.
2. Se filtran los del nodo `model`: la salida de las herramientas no llega al cliente.
3. Cada evento `data:` lleva un JSON con el campo `content`.
4. El primer texto llega antes; el tiempo total es el mismo.

SSE en el tiempo

Diagrama de secuencia



Solo el nodo model. Los trozos del nodo `tools` se descartan; cada trozo del nodo `model` sale como un evento `data:`.

Pontificia Universidad JAVERIANA Colombia

Operar

LLMOps

Qué vigilar en producción

LLMOps

Dimensión	Cómo se mide en el curso
Costo	Tokens de entrada y salida por llamada; US$ por corrida
Calidad	Experimentos de LangSmith con dataset, código y juez (semana 6)
Trazas	LangSmith con `tracing_context` y un presupuesto de trazas
Regresiones	CI con pruebas sin modelo y arranque de `langgraph dev`
Seguridad	Herramientas de solo lectura, aprobación humana, claves fuera del código

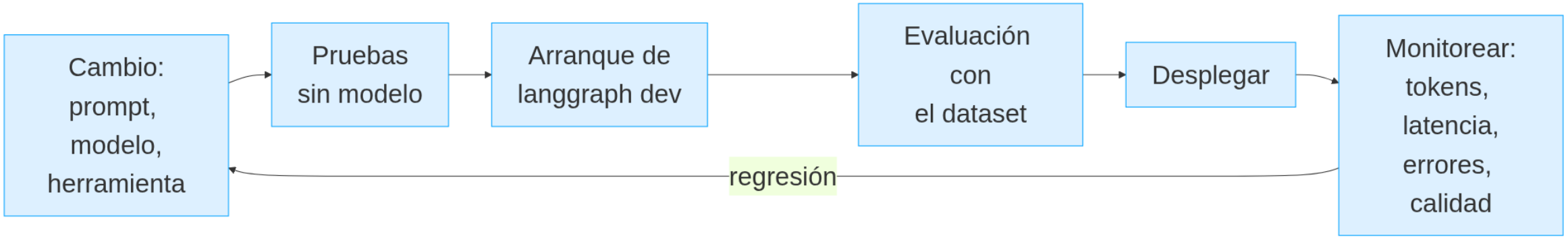
Costo mensual estimado

Con los turnos medidos en el skill deploy-app

Turnos al mes	Tokens (entrada / salida)	Costo estimado
1 turno promedio	≈ 718 / 50	≈ US$0,0001
1.000	≈ 0,7 M / 0,05 M	≈ US$0,10
10.000	≈ 7,2 M / 0,5 M	≈ US$1
100.000	≈ 72 M / 5 M	≈ US$10

El ciclo de una versión

LLMOps



Antes y después de desplegar. Cada cambio pasa por pruebas, arranque y evaluación antes de desplegar; el monitoreo dice cuándo volver a empezar.

Lo que esta demo no hace

Antes de exponerla

Límites conocidos

1. No tiene autenticación ni control de dueño por conversación.
2. No serializa turnos simultáneos sobre el mismo `chat_id`.
3. Las credenciales de PostgreSQL son de demostración y el servicio escucha en loopback.
4. Cada llamada usa un modelo de pago: medir antes de abrirla a más usuarios.

Antes de exponer la API

Checklist de seguridad

Control	Por qué
Autenticación	Saber quién llama
Dueño de cada hilo	Que nadie lea una conversación ajena adivinando su `chat_id`
Límite de solicitudes por usuario	Contener abuso y costo
Tamaño máximo del mensaje y de la conversación	Acotar tokens por turno
HTTPS detrás de un proxy	Cifrar el tráfico; no exponer uvicorn directo
Secretos fuera del código y de los logs	Una fuga de la clave cuesta dinero
Herramientas de solo lectura o con aprobación	El modelo puede pedir cualquier cosa
Respaldos de la base y dependencias al día	Recuperar conversaciones; cerrar vulnerabilidades

InMemorySaver frente a PostgresSaver

Dónde vive la memoria

	InMemorySaver	PostgresSaver
Dónde guarda	En la memoria del proceso	En tablas de PostgreSQL
Tras reiniciar	Se pierde todo	Los hilos siguen ahí
Varios procesos	Cada uno con su memoria	Comparten la misma base
Para qué	Notebooks, pruebas, demos	Un servicio que usan personas
Cuidado	Nada que respaldar	Respaldar el volumen; borrarlo borra conversaciones

Pontificia Universidad JAVERIANA Colombia

Laboratorio

API local y LangGraph Studio

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. Levantar PostgreSQL con Docker Compose y la API con uvicorn.
2. Conversar con dos `chat_id` distintos y reiniciar la API entre turnos.
3. Abrir el mismo agente en LangGraph Studio (`acme`).
4. Inspeccionar los checkpoints guardados con `docs/sql/checkpoints.sql`.

Las ideas que nos llevamos del curso

Cierre

Criterios de construcción

1. El modelo es un componente: la aplicación prepara el contexto y comprueba la salida.
2. Las respuestas se sustentan en herramientas y documentos citados.
3. Elegir el patrón más simple; los controles van en el código.
4. Medir calidad, latencia y costo antes de decidir.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 9 y 10	Despliegue y pruebas
AI Engineering, cap. 10	Arquitectura y feedback de usuarios
Documentación de LangGraph	Persistence, deployment y Studio

Costo del curso por semana

Semanas 1 a 6 · llamadas al recorrer los notebooks

Semana	Notebooks	Llamadas	Costo estimado
1	L01, L02	26	≈ US$0,005
2	L03	4	≈ US$0,001
3	L04, L05	14	≈ US$0,003
4	L06, L07, L08	3 (embeddings locales)	≈ US$0,001
5	L09, L10	18	≈ US$0,003
6	L11	38 + 11 trazas	≈ US$0,007

Costo del curso por semana

Semanas 7 a 12 · llamadas al recorrer los notebooks

Semana	Notebooks	Llamadas	Costo estimado
7	L12, L13	≈ 7	≈ US$0,001
8	L14 a L19	15	≈ US$0,003
9	L20	≈ 4	≈ US$0,001
10	L21	≈ 5	≈ US$0,001
11	L22	2 a 6	≈ US$0,001
12	API local	1 o 2 por turno	≈ US$0,0001 por turno

Glosario del curso (1/4)

Modelos y LangChain

Término	Significado
Token	Unidad de texto que procesa el modelo; se paga por token
Ventana de contexto	Todo lo que el modelo lee en una llamada
Prompt de sistema	Instrucciones generales que orientan cada respuesta
Plantilla	Texto con huecos que se llenan con datos (ChatPromptTemplate, Jinja)
Salida estructurada	Respuesta validada contra un esquema de Pydantic
Runnable	Componente con invoke, batch y stream que se conecta con |
Herramienta (tool)	Función que el modelo puede pedir y nuestro código ejecuta
Function calling	El modelo propone qué función llamar y con qué argumentos

Glosario del curso (2/4)

Agentes y RAG

Término	Significado
Agente	Un modelo que usa herramientas en un ciclo hasta responder
Embedding	Vector que representa el significado de un texto
Fragmento (chunk)	Trozo de un documento que se indexa y se recupera
Vector store	Índice que busca los vectores más cercanos (FAISS, Chroma)
Retriever	Componente que devuelve los k fragmentos para una pregunta
RAG	Recuperar fragmentos y responder solo con ellos, citando la fuente
BM25	Búsqueda por palabras exactas
RRF	Fusión de rankings: suma 1 / (60 + puesto) de cada lista

Glosario del curso (3/4)

Memoria, evaluación y MCP

Término	Significado
Checkpointer y thread_id	Guarda el estado de cada conversación bajo su identificador
Traza	Registro de los pasos de una ejecución en LangSmith
Dataset de evaluación	Preguntas con respuestas de referencia escritas por personas
Evaluador de código	Regla exacta que puntúa una respuesta sin modelo
Juez (LLM-as-judge)	Un modelo que puntúa con una rúbrica; se calibra contra personas
Experimento	Una configuración evaluada sobre un dataset
MCP	Protocolo para que las apps descubran y usen servidores de herramientas y datos
Elicitation	Un servidor MCP le pregunta un dato a la persona

Glosario del curso (4/4)

LangGraph y despliegue

Término	Significado
StateGraph	Grafo de nodos que comparten un estado
Reducer	Regla que combina una actualización con el estado
Gate y router	Decisiones en código o del modelo que eligen el siguiente nodo
Send	Lanza un nodo por elemento, cada uno con su entrada
Subgrafo	Un grafo compilado usado como nodo
interrupt	Pausa el grafo hasta que una persona decide
SSE	Eventos del servidor: la respuesta llega por trozos
LLMOps	Operar una app con LLMs: costo, trazas, evaluación y seguridad



¡Gracias!

Todo el material del curso, los notebooks y las presentaciones están en el repositorio.

Sergio Alberto Mora Pardo - H1 2027

GitHub · LinkedIn · Repositorio del curso